

30224 – Information Systems

Practice 3: Design, development and implementation of a Web information system: the model layer

2025-09-25

Table of contents

1	Objectives	2
2	Contents	3
2.1	Lab agenda (recommended order of tasks)	3
2.2	Reference architecture and tools	3
3	Block 1: Setting up the development environment	4
3.1	Installing the virtual machine (optional)	4
3.2	Setting up the workspace	6
3.3	Creating the first Jakarta EE web application (baseline test)	6
3.4	DBMS installation	7
3.4.1	Run PostgreSQL with Docker (single container)	8
3.4.2	Docker Compose (recommended)	8
3.5	Accessing PostgreSQL	9
4	Block 2: Designing and creating Tables in a Relational DBMS	11
4.1	Example: ER model	11
4.2	Example: Tables creation (PostgreSQL + PostGIS)	11
5	Block 3: Data layer development	14
5.1	Value Object (VO) pattern	14

5.2	Data Access Object (DAO) pattern	14
5.3	Learning outcomes	15
5.4	Recommended implementation steps	15
5.5	Transactions and consistency	15
5.6	Testing the data layer	15
5.7	What to include in the report	16
5.8	Rubric expectations for this block	16
5.9	Minimal end-to-end example (sketch)	16
5.10	Running the SQL scripts	17
5.11	Seed and verify	17
5.12	Troubleshooting	18
6	Suggested scope/timebox	18
7	Exit checklists (before submission)	18
8	Practice submission	19
9	Grading criteria	19
9.1	Minimum requirements	19
9.2	Additional considerations	20
10	Additional resources	20

1 Objectives

Web applications run on at least one web server and are accessed through general-purpose clients (e.g., a web browser). They usually serve static content (HTML, PDF) and dynamic content generated in response to user requests. Dynamic content is produced by the application container/server, which in turn accesses data in one or more data sources.

This practice focuses on using a Database Management System (DBMS) to design and implement the persistence model and the data access layer of your application.

The specific objectives are:

1. **Install/Configure** the selected **DBMS**.
2. **Design** an **Entity–Relationship model** for the application’s data.
3. **Transform** the ER model into a logical **model** (relational or other, depending on the DBMS).
4. **Implement** the **persistence model** using the chosen DBMS.
5. **Design and implement** the application’s **data access layer** (e.g., JDBC with DAO/VO in Java).

6. **Prepare a report** covering design, methodology, decisions, team distribution, difficulties, and cost in worked hours.

i Note

The reference environment includes **Java 21 LTS**, **Jakarta EE 10/11**, **Tomcat 10.1/11**, and **PostgreSQL 17**. Alternatives such as **Nest.js** (Node.js) or **Django** (Python) are acceptable.

2 Contents

The practice is divided into three main blocks:

1. **Setup of the development environment:** VM and tools installation (optional), workspace setup, starting PostgreSQL with Docker/Compose, connecting and initializing database, schema, and PostGIS.
2. **Database design and creation:** ER design, transformation to a logical model, and creation of tables and constraints in the DBMS.
3. **Data layer development:** Implementation of the DAO/VO layer (e.g., JDBC) to interact with the DBMS.

Additionally, you must submit a brief report summarizing decisions, methodology, time spent, and other relevant information.

2.1 Lab agenda (recommended order of tasks)

1. Kickoff: objectives, stack recap, and review of the reference architecture/tools.
2. Start PostgreSQL (Docker single container or Compose).
3. Connect to PostgreSQL (psql from host or inside container; DBeaver optional).
4. Initialize: create project database, schema, and enable PostGIS.
5. Design the ER model and transform it into the logical model (Block 2).
6. Create tables and other database objects (adapt the provided DDL).
7. Implement the data access layer (DAO/VO) and verify CRUD (Block 3).
8. Record decisions and effort; prepare the report (methodology, decisions, difficulties).
9. Package and submit the deliverables.

2.2 Reference architecture and tools

Use a coherent stack suitable for your team's skills and project goals. The following stack is recommended:

- **PostgreSQL 17** (or later).
- **Java 21 LTS** with **Jakarta EE 10/11**.
- Development with Servlets/JSPs following **MVC** and **DAO/VO** patterns.
- **Tomcat 10.1/11** (Jakarta Servlet container).
- Optional IDEs and tools: **Eclipse IDE**, **DBeaver**, **Docker**.

Note: Topics related to the view layer design and **Git** workflow are handled in Practice 2 and are intentionally omitted here.

3 Block 1: Setting up the development environment

3.1 Installing the virtual machine (optional)

If you prefer a reproducible environment, install **VirtualBox** and create a Linux VM (e.g., Ubuntu LTS). Download from <https://www.virtualbox.org/wiki/Downloads>.

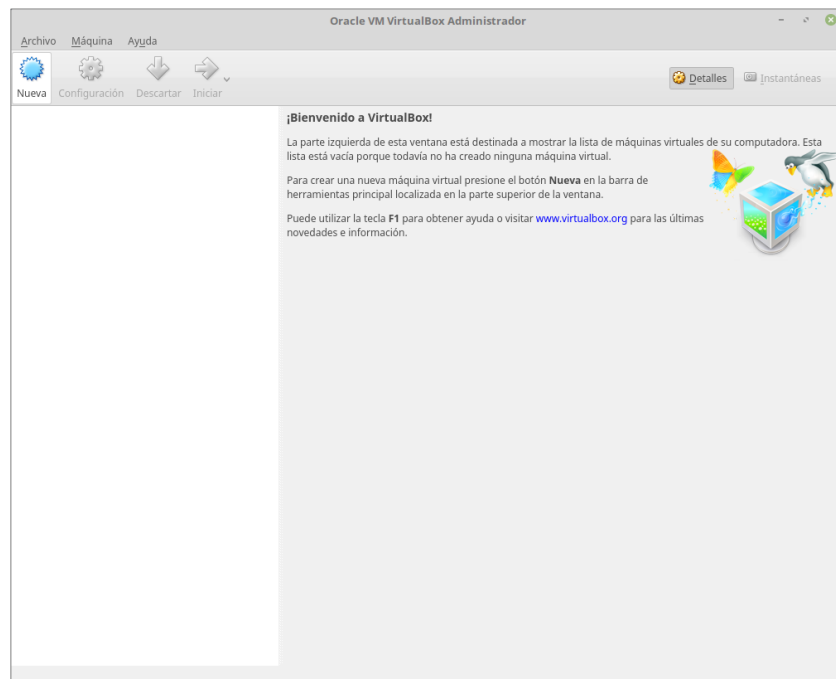


Figure 1: Virtual Box Welcome Screen

Choose a name and select Linux Ubuntu (64-bit).



Figure 2: Selection of Linux Ubuntu (64-bit)

Assign memory according to your system (1 GB; 4 GB if you have 8 GB RAM).



Figure 3: Setting a memory size

Create a virtual hard drive and start the VM.

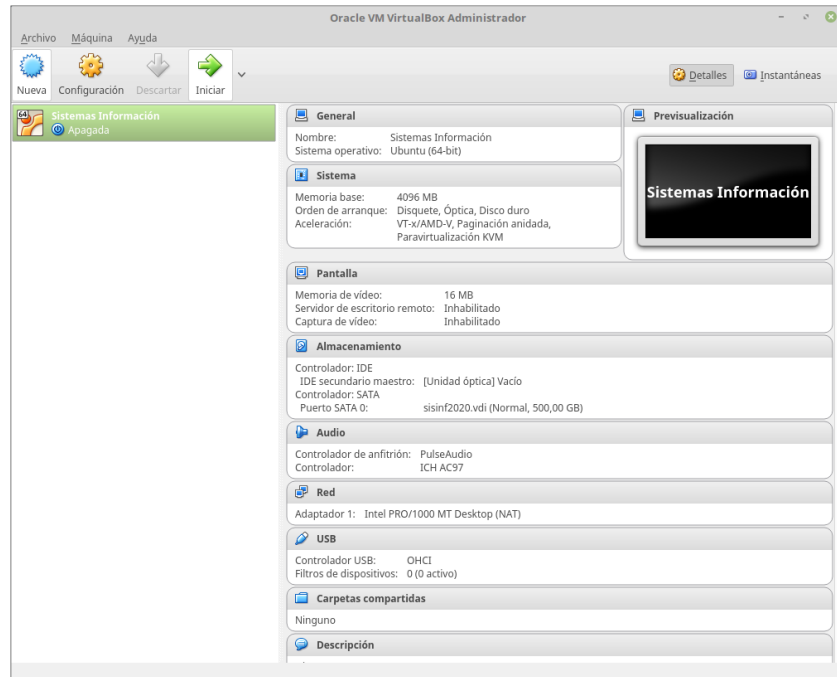


Figure 4: Configuration Summary

Recommended software to install in the VM (or natively):

- Linux Lite or Ubuntu LTS
- Java: OpenJDK 21 (Java 21 LTS)
- PostgreSQL 17
- Eclipse IDE for Enterprise Java and Web Developers
- DBeaver (standalone or Eclipse plugin)
- Tomcat 10.1 or 11 (Jakarta Servlet container)
- Firefox and Chrome
- Docker

3.2 Setting up the workspace

Use the default workspace that Eclipse suggests, or choose a consistent directory where you keep all projects for these practices.

3.3 Creating the first Jakarta EE web application (baseline test)

1. In Eclipse, choose File → New → Dynamic Web Project.

2. Name the project (e.g., `test_project`). Click Next twice and check “Generate web.xml deployment descriptor”. Finish.

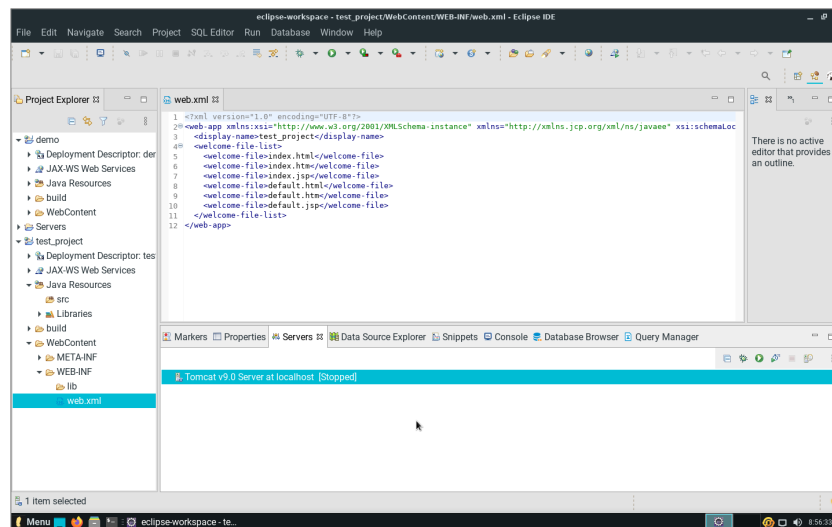


Figure 5: Project structure

3. In the `webapp` folder, create an `index.html` with a simple page such as `<h1>Hello, World!</h1>`.
4. In the Servers view, configure Tomcat 10.1 or 11. Ensure “Use Tomcat installation (takes control of Tomcat installation)” is selected, then start the server.
5. Right-click the project → Run As → Run on Server. Eclipse will build and deploy the WAR to Tomcat.

Access the application at:

`http://localhost:8080/test_project`

If you use Tomcat 10+/Jakarta EE, ensure imports use `jakarta.*` (not `javax.*`).

3.4 DBMS installation

- **PostgreSQL** is the recommended DBMS. A convenient way to run it locally is via **Docker** images from [Docker Hub](#): [postgres](#).
- Cloud-hosted DBMS services are also acceptable and simplify collaboration.
- For free cloud DBMS options, see [this list](#).
- The database installed on Hendrix can also be used when available.

3.4.1 Run PostgreSQL with Docker (single container)

```
# Pull PostGIS image for PostgreSQL 17
docker pull postgis/postgis:17-3.4

# Create a named volume for durable storage
docker volume create pgdata_pr3

# Start the container
docker run -d \
  --name pr3-postgres \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=postgres \
  -e POSTGRES_DB=postgres \
  -p 5432:5432 \
  -v pgdata_pr3:/var/lib/postgresql/data \
  postgis/postgis:17-3.4
```

3.4.2 Docker Compose (recommended)

Create `compose.yaml` alongside your project and run it with Docker Compose.

Official image (recommended):

```
services:
  db:
    image: postgis/postgis:17-3.4
    container_name: pr3-postgres
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: postgres
    volumes:
      - pgdata_pr3:/var/lib/postgresql/data

volumes:
  pgdata_pr3:
```

Context for the alternative image: If you hit architecture issues (e.g., Apple Silicon M-series) with certain images or tooling, you can use the `kartoza/postgis` image. Note

that its environment variables differ (e.g., `POSTGRES_PASS` instead of `POSTGRES_PASSWORD`) and it offers helpers like `ALLOW_IP_RANGE` and `POSTGRES_MULTIPLE_EXTENSIONS`.

Alternative image (kartoza):

```
services:
  db:
    image: kartoza/postgis
    container_name: pr3-postgres
    ports:
      - "5432:5432"
    environment:
      - POSTGRES_USER=postgres
      - POSTGRES_PASS=postgres
      - POSTGRES_DB=postgres
      - ALLOW_IP_RANGE=0.0.0.0/0
      - >-
        POSTGRES_MULTIPLE_EXTENSIONS=postgres,hstore,postgis_topology,
        postgis_raster,pgrouting
    volumes:
      - pgdata_pr3:/var/lib/postgresql/data

volumes:
  pgdata_pr3:
```

Start the stack:

```
docker compose up -d
```

3.5 Accessing PostgreSQL

From a terminal on the machine where PostgreSQL runs:

```
psql -U postgres -W
```

If using Docker:

```
# Open psql inside the container (avoid peer auth issues)
docker exec -it --user postgres pr3-postgres psql -U postgres

# Or connect from the host via connection string
```

```
psql postgresql://postgres:postgres@localhost:5432/postgres

# Initialize database and schema (examples)

docker exec -it --user postgres pr3-postgres psql -U postgres -d
postgres -c "CREATE DATABASE sisinf_grupo_x;"
docker exec -it --user postgres pr3-postgres psql -U postgres -d
sisinf_grupo_x -c "CREATE SCHEMA IF NOT EXISTS sisinf_p3;"
docker exec -it --user postgres pr3-postgres psql -U postgres -d
sisinf_grupo_x -c "CREATE EXTENSION IF NOT EXISTS postgis WITH SCHEMA
sisinf_p3;"

# Or connect using docker

docker exec -it --user postgres pr3-postgres psql -U postgres -d
postgres
```

For a graphical interface, use **DBeaver** ([website](#)).

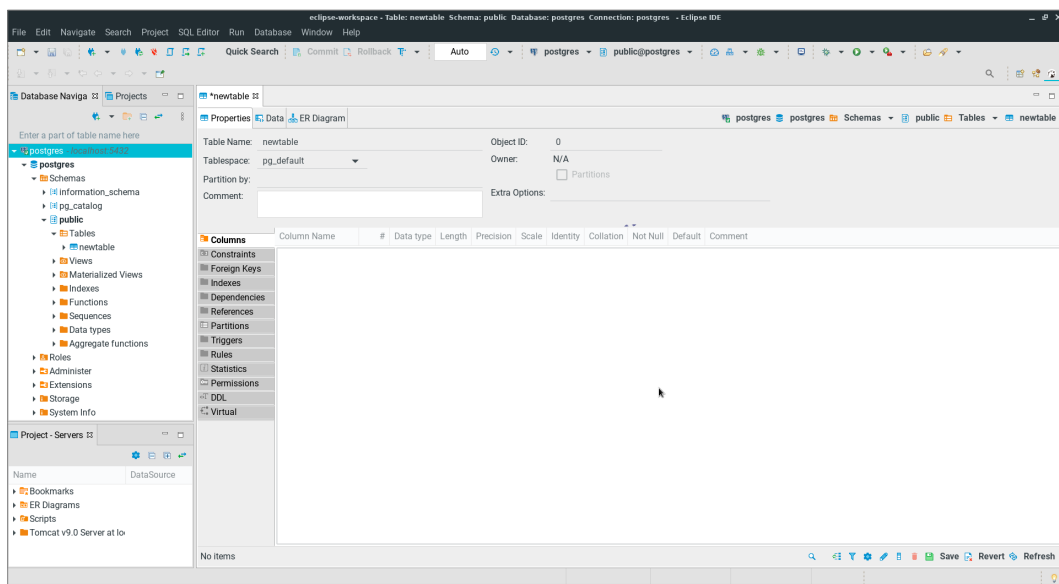


Figure 6: DBeaver database IDE

Ensure that remote access is configured correctly by editing `pg_hba.conf` as described in the official [PostgreSQL documentation](#).

Create a dedicated **database** and **schema** for your project. Avoid using the **public** schema for application tables.

4 Block 2: Designing and creating Tables in a Relational DBMS

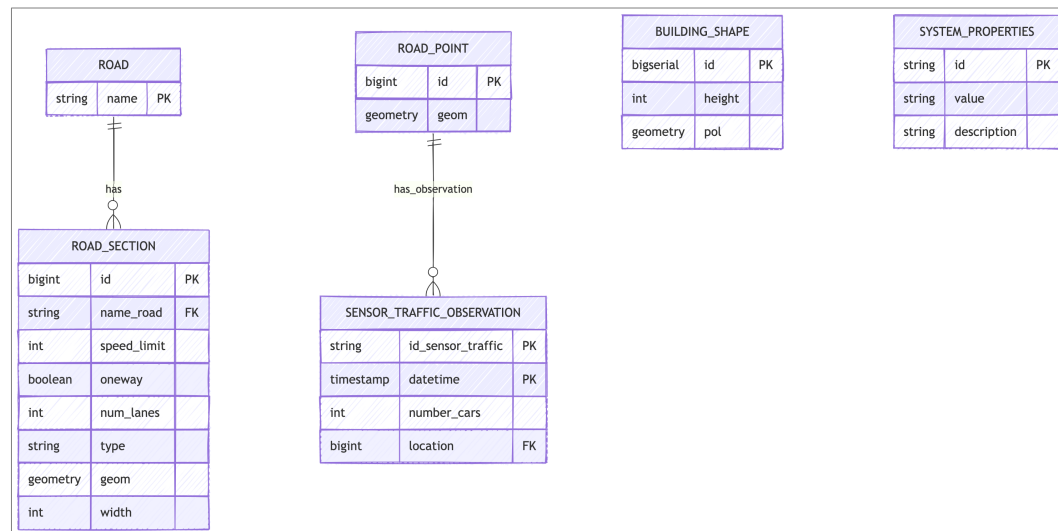
Before implementing tables, start with the design and transformation of the data model:

- Design the Entity–Relationship (ER) model for the database required to store the data of the Web application you are building.
- Transform the ER model into a relational model (if a relational DBMS is selected) or into the data model used by the selected DBMS (e.g., network, hierarchical, graph).
- Finally, implement the data persistence model in the selected DBMS.

4.1 Example: ER model

We model a simple urban road network and traffic monitoring scenario. Named roads are split into sections with attributes such as speed limit, type, number of lanes, oneway, and geometry. Discrete points on the network act as reference anchors for sensors and operations. Buildings are represented by polygon footprints and a height attribute. Traffic sensors produce time-stamped observations of vehicle counts and are linked to a specific road point. A small system properties table stores configurable key–value pairs used by the application.

The following ER sketch aligns with the DDL used in the next section.



4.2 Example: Tables creation (PostgreSQL + PostGIS)

The following example shows basic types and geospatial data using **postgis**. Adapt names and constraints to your domain.

```

-- Database and schema (adjust names)
CREATE DATABASE sisinf_grupo_x;

CREATE SCHEMA IF NOT EXISTS sisinf_p3;

-- Enable PostGIS in the project schema
CREATE EXTENSION IF NOT EXISTS postgis WITH SCHEMA sisinf_p3;

-- Make PostGIS types (e.g., geometry) visible without schema
qualification
SET search_path TO sisinf_p3, public;

-- Drop existing tables if present (order matters due to FKs)
DROP TABLE IF EXISTS sisinf_p3.sensor_traffic_observation;
DROP TABLE IF EXISTS sisinf_p3.road_section;
DROP TABLE IF EXISTS sisinf_p3.building_shape;
DROP TABLE IF EXISTS sisinf_p3.road_point;
DROP TABLE IF EXISTS sisinf_p3.road;
DROP TABLE IF EXISTS sisinf_p3.system_properties;

-- Properties table
CREATE TABLE sisinf_p3.system_properties (
    id          varchar(255) PRIMARY KEY,
    value       varchar(255),
    description varchar(255)
);

-- Roads and sections
CREATE TABLE IF NOT EXISTS sisinf_p3.road (
    name varchar(255) PRIMARY KEY
);

CREATE TABLE IF NOT EXISTS sisinf_p3.road_section (
    id          bigint PRIMARY KEY,          -- OSM way id, for
    example
    name_road   varchar(255) NOT NULL,
    speed_limit int NOT NULL,                -- km/h
    oneway      boolean DEFAULT false,
    num_lanes   int,
    type        varchar(255) NOT NULL,
    geom        geometry(LINESTRING, 4326) NOT NULL,
    width       int,
    CONSTRAINT road_section_road_fk FOREIGN KEY (name_road) REFERENCES
    sisinf_p3.road(name)

```

```

);

-- Points on the road network (e.g., OSM node id)
CREATE TABLE IF NOT EXISTS sisinf_p3.road_point (
    id    bigint PRIMARY KEY,
    geom  geometry(POINT, 4326) NOT NULL
);

-- Building footprints
CREATE TABLE IF NOT EXISTS sisinf_p3.building_shape (
    id      bigserial PRIMARY KEY,
    height  int NOT NULL CHECK (height > 0),
    pol     geometry(POLYGON, 4326) NOT NULL
);

-- Traffic sensor observations
CREATE TABLE IF NOT EXISTS sisinf_p3.sensor_traffic_observation (
    id_sensor_traffic varchar(255),
    datetime          timestamp,
    number_cars        int,
    location           bigint,
    CONSTRAINT traffic_observation_pk PRIMARY KEY (id_sensor_traffic,
    datetime),
    CONSTRAINT sensor_location_fk FOREIGN KEY (location) REFERENCES
    sisinf_p3.road_point(id)
);

```

Tip

Run the DDL in the target database (e.g., `sisinf_grupo_x`). If you are in `psql`, connect first with `\c sisinf_grupo_x` before executing the script.

You may use a graphical editor like **DBeaver** (see Figure 6) to create and manipulate tables. Prefer a dedicated schema rather than `public`.

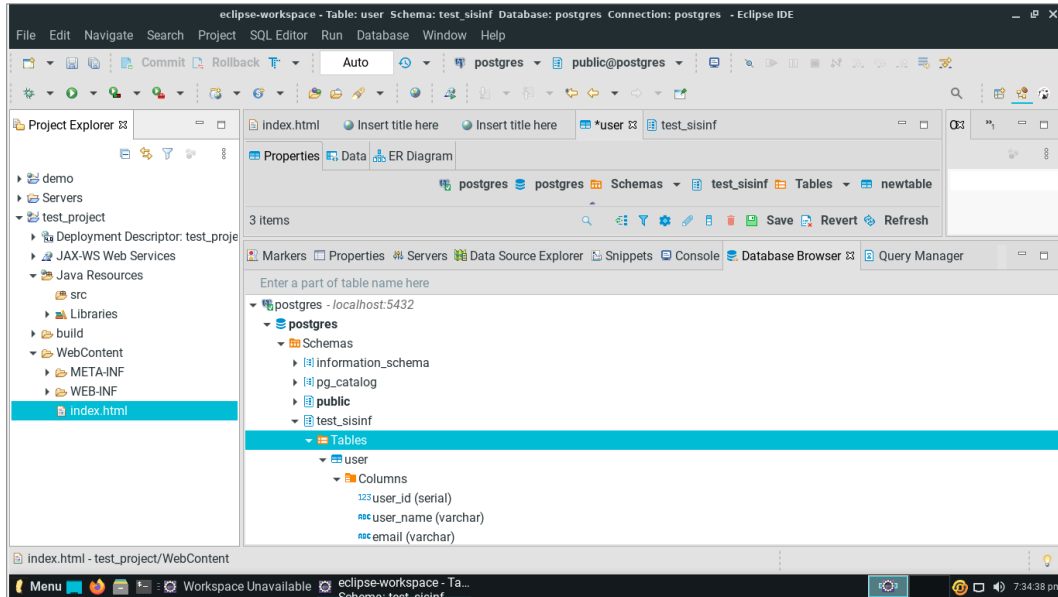


Figure 7: Creating tables from the IDE

In addition to tables in the chosen schema, create users and assign only the access permissions required by your application. Also add indexes on frequently joined foreign keys to improve query performance.

5 Block 3: Data layer development

With the logical model implemented in the DBMS, now implement the application's data access layer.

5.1 Value Object (VO) pattern

The **Value Object** (VO, also known as Transfer Object) pattern groups multiple property values into a single object. This improves readability and makes data transfer between layers explicit. In Java, VO classes typically implement `Serializable`. See the official [Java transfer object overview](#).

5.2 Data Access Object (DAO) pattern

The **Data Access Object** (DAO) pattern abstracts persistence from business logic, allowing the latter to delegate data operations to DAO classes. This eases changes in the data source without modifying business logic. See the official [DAO overview](#).

When using a relational DBMS with Java, **JDBC** is a suitable low-level API for DAOs. Alternative stacks (e.g., Node.js, Python) should choose equivalent data access libraries and apply the same separation of concerns.

5.3 Learning outcomes

- Understand the purpose of separating concerns between business logic and persistence.
- Apply the **DAO/VO** patterns to organize data access code.
- In Java, Use **JDBC** safely (connections, prepared statements, transactions, exceptions, and resource management).

5.4 Recommended implementation steps

1. Identify one domain entity you already modeled in the database (for example, a table from Block 2).
2. Define its VO: fields must map 1:1 to the table columns that are relevant to the application logic.
3. Define a DAO interface: include create, read (by id), read (list or search), update, and delete as needed.
4. Implement the DAO using JDBC: acquire connections from a DataSource, use prepared statements, and map ResultSets to VO instances.
5. Add error handling that preserves root causes and converts SQL exceptions into meaningful application exceptions.
6. Write small tests that exercise at least one full CRUD cycle end-to-end.

5.5 Transactions and consistency

- Group related writes in a single transaction to keep data consistent.
- Set auto-commit to false for multi-step operations, commit on success, and rollback on failure.
- Choose a sensible isolation level; the default is often sufficient for this practice.

5.6 Testing the data layer

- Use a real database for tests (the Docker setup in Block 1). Clean the schema between tests.
- Seed minimal data for read tests; assert on both data values and row counts.
- Include at least one negative test (e.g., foreign key violation) and verify error handling.

5.7 What to include in the report

- A short description of your DAO/VO structure and responsibilities.
- The main queries used and any important constraints or indexes that support them.
- How you manage connections, transactions, and errors.
- Evidence of testing (what was tested and how).

5.8 Rubric expectations for this block

- Correct application of DAO/VO patterns and clear separation from business logic.
- Readable, maintainable data access database code.
- Basic transactions used where appropriate; resources are always released.
- Tests demonstrating CRUD and handling of typical failures.

5.9 Minimal end-to-end example (sketch)

Value Object (VO):

```
public final class RoadSection {
    private final long id;
    private final String roadName;
    private final int speedLimit;

    public RoadSection(long id, String roadName, int speedLimit) {
        this.id = id;
        this.roadName = roadName;
        this.speedLimit = speedLimit;
    }
    public long getId() { return id; }
    public String getRoadName() { return roadName; }
    public int getSpeedLimit() { return speedLimit; }
}
```

DAO interface (excerpt):

```
public interface RoadSectionDao {
    Optional<RoadSection> findById(long id) throws SQLException;
}
```

DAO via JDBC (excerpt):

```

public final class JdbcRoadSectionDao implements RoadSectionDao {
    private final DataSource dataSource;
    public JdbcRoadSectionDao(DataSource ds) { this.dataSource = ds; }

    @Override
    public Optional<RoadSection> findById(long id) throws SQLException {
        final String sql = "SELECT id, name_road, speed_limit FROM
        sisinf_p3.road_section WHERE id = ?";
        try (Connection c = dataSource.getConnection();
            PreparedStatement ps = c.prepareStatement(sql)) {
            ps.setLong(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) return Optional.empty();
                return Optional.of(new RoadSection(
                    rs.getLong("id"),
                    rs.getString("name_road"),
                    rs.getInt("speed_limit")));
            }
        }
    }
}

```

5.10 Running the SQL scripts

Execute the DDL against the target database:

```

psql postgresql://postgres:postgres@localhost:5432/sisinf_grupo_x -f
path/to/ddl.sql

```

With Docker:

```

docker exec -it --user postgres pr3-postgres psql -U postgres -d
sisinf_grupo_x -f /scripts/ddl.sql

```

5.11 Seed and verify

```

INSERT INTO sisinf_p3.road(name) VALUES ('N-330');
INSERT INTO sisinf_p3.road_section(id, name_road, speed_limit, type,
geom)

```

```
VALUES (1001, 'N-330', 90, 'primary', ST_GeomFromText('LINESTRING(0 0, 1
1)', 4326));

SELECT id, name_road, speed_limit FROM sisinf_p3.road_section WHERE id =
1001;
```

5.12 Troubleshooting

- Geometry type not found: ensure you run `CREATE EXTENSION postgis ...` in `sisinf_grupo_x` and set `SET search_path TO sisinf_p3, public;` before creating tables.
- SRID mismatch: ensure geometries use the same SRID (here 4326). If needed, wrap literals with `ST_SetSRID(..., 4326)` or transform inputs with `ST_Transform(geom, 4326)` before insert/update.
- Peer authentication failed inside container: run `docker exec ... --user postgres ...` or connect via TCP with password from host.
- Port 5432 already in use: stop the conflicting service or map another host port (e.g., `-p 55432:5432`).
- Container healthy check: `docker ps` and `docker logs pr3-postgres`.

6 Suggested scope/timebox

- Aim to fully implement 1 entity (schema + DAO/VO + CRUD + test) within each lab time.

7 Exit checklists (before submission)

- Database is running and accessible (Docker or Compose).
- Project database, schema, and PostGIS extension created.
- Core tables created with primary and foreign keys.
- At least one DAO/VO implemented and tested (CRUD).
- `sources/scripts` includes DDL/DML and init steps used.
- `MemoriaPractica3.pdf` completed (design, methodology, costs, difficulties).

Example of `sources/scripts/`:

```
sources/scripts/
ddl.sql
seed.sql
README.md    # how to run: psql -f ddl.sql; psql -f seed.sql
```

DAO/VO quality checklist:

- DataSource (pool), prepared statements, try-with-resources
- Minimal, clear mapping logic (column $\rightarrow$ field)
- Meaningful exception handling (preserve causes)

8 Practice submission

The practice is completed in groups of three. Submit a `.tar` or `.zip` named `practice3_NIA1_NIA2_NIA3.{tar|zip}` (where NIA1, NIA2, and NIA3 are the Student Identification Numbers) with the following contents:

1. `sources/scripts` directory with the scripts used to create tables and define DB objects in the selected DBMS.
2. `MemoriaPractica3.pdf` (max 10 pages) including:
 - ER model and logical model design.
 - Data persistence layer design (classes or code implemented).
 - Methodology (tools, task distribution, decisions, planning).
 - Cost (hours spent, split by database setup and data layer development).
 - Difficulties encountered.

The `.tar` or `.zip` file must be submitted via the University of Zaragoza's [Moodle](#) platform. The submission deadline is the day before the fifth lab session (Practice 4), 16th October at 11:59 PM.

9 Grading criteria

9.1 Minimum requirements

- Correct ER model syntax and design.
- Transformation of the ER model into the selected logical model (e.g., relational).
- Implementation of the logical model in the DBMS (appropriate types, indexing, primary keys, foreign keys, etc.).
- Clear, consistent data access layer design using appropriate patterns and code style for the chosen technology.

9.2 Additional considerations

- Quality of the report (structure, writing, punctuation, index, references).
- Justification of design decisions.
- Methodology and cost breakdown.
- Notable aspects and innovations.

10 Additional resources

- PostgreSQL authentication (pg_hba.conf): [Official docs](#)
- PostgreSQL connection URIs and psql basics: [Official docs](#)
- PostGIS documentation (geometry, SRID 4326, functions): [Documentation index](#)
- PostGIS installation and search_path notes: [Installation guide](#)
- Query planning and performance (EXPLAIN): [Using EXPLAIN](#)
- Transactions and isolation levels: [Official docs](#)
- JDBC (DataSource, try-with-resources): [Oracle JDBC tutorial](#)
- Connection pooling: [HikariCP](#)
- SQL Injection prevention (prepared statements): [OWASP Cheat Sheet](#)
- Docker image: [Docker Hub: postgres](#)
- Compose file reference: [Docker docs](#)
- Testing the data layer (Java): [Testcontainers PostgreSQL module](#)